Tips and tricks around TEI Publisher 10

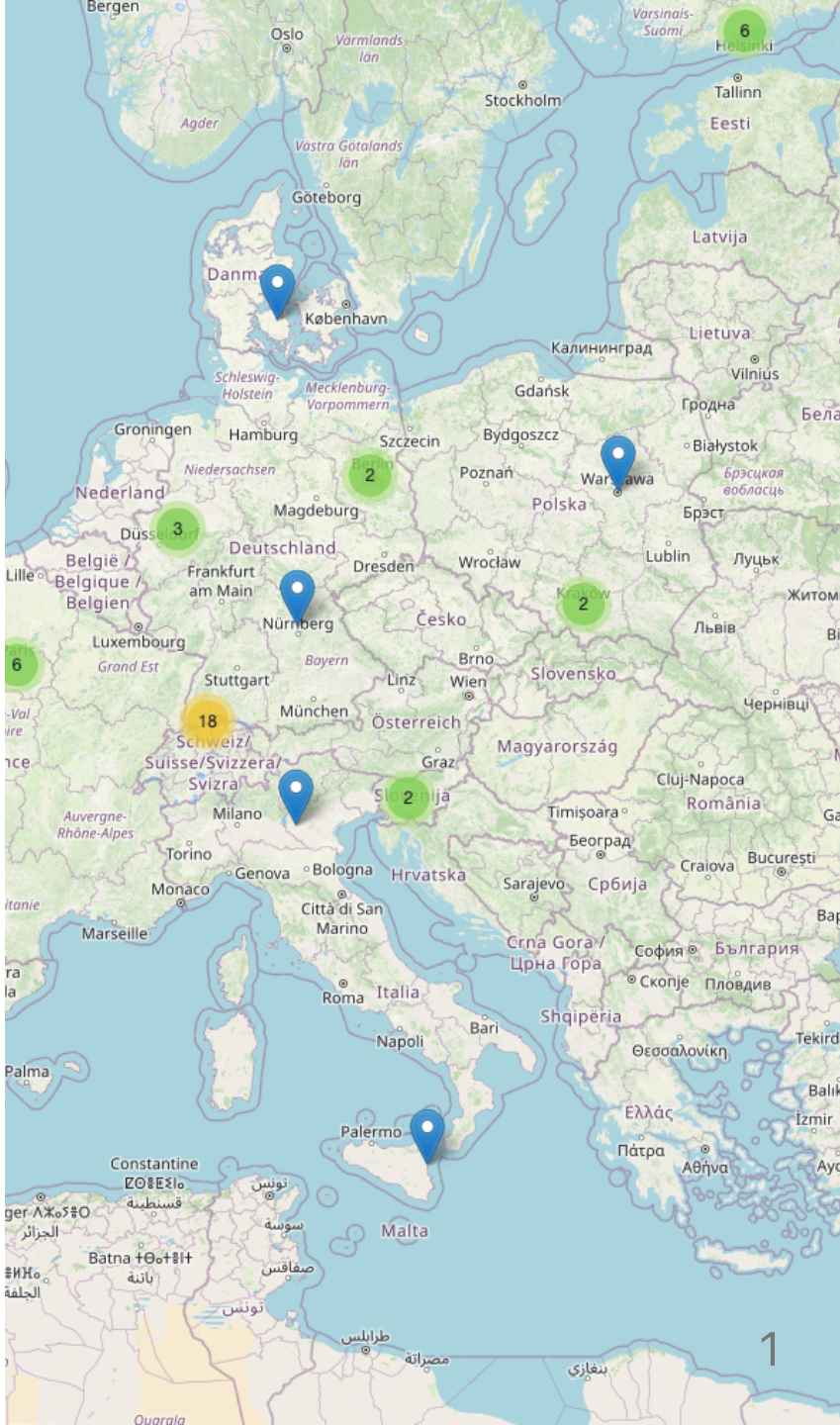
e-editiones community meetup April 7th, 2026

App Customization

Before adding code, ask:

> *Can this be done in* `config.json`*?*

- All profiles come with configuration settings
- JSON Schema (`jinks.json`) is ultimate source of truth
- Check **Merged Configuration** dropdown in Jinks for all config options



Report Back

If there's no configuration setting yet:

- Should there be one?
- If yes, consider opening an issue / PR or ask on slack

Don’t Fight the Generator

Avoid changing files generated by Jinks

Instead: add your own CSS, scripts, and API pieces as **separate files**, and **register** them in `config.json` or in **HTML template frontmatter**.

CSS

Use `styles` array to add custom CSS files

```
{
    "styles" : [ "resources/css/shakespeare-theme.css" ]
}
```

Best practice: prefix CSS filename with project abbrev and save in `resources/css`.

Javascript

Use `script.custom`:

```
{
    "script": {
        "custom": [
            "resources/scripts/shakespeare-facets.js"
        ]
    }
}
```

Open API Endpoints (XQuery)

Add `.json` and corresponding `.xql` implementation to `modules` and register in `config.json`:

```
{
    "api" : [
        {
            "spec" : "shakespeare-api.json",
            "prefix" : "shakes",
            "id" : "https://e-editiones.org/api/shakespeare",
            "path" : "shakes-api.xql"
        }
    ]
}
```

HTML Templates

- Default template defined in `defaults.template`
- Start by copying `templates/pages/basic.html` for document views
- Respect the template inheritance hierarchy
- Avoid overwriting `content.html` or `base.html`
- Stick to the default page layout areas for arranging content on the page

Using Frontmatter

Most settings can also be changed in HTML frontmatter:

```
<template>
    ---json
    {
        "templating": {
            "extends": "templates/layouts/base.html"
        },
        "styles": [ "resources/css/landing-page.css" ]
    }
    ---
    <p>Page content</p>
</template>
```

Custom HTML Blocks

Inject custom HTML blocks into pages with `templating.use`:

```
"templating" : {
    "use" : [ "templates/iiif-blocks.html" ]
}
```

If a conflict cannot be avoided …

- Sometimes you need to overwrite a file, e.g. `collection.xconf`
- If you are sure about it: add it to the `ignore` list in `config.json`:

```
{
    "ignore" : [ "collection.xconf", "index.xql" ],
}
```

Development Workflow

1. create base application via Jinks
2. synchronize to local directory
3. commit to version control

When to Regenerate?

- `config.json` changed
- CSS related to webcomponents changed
- color palette changed

When to Sync?

- whenever you change something in the database (but not locally)
- in particular:
  - ODD changes via editor
  - after Jinks regenerated app

Sync Options: eXist-db -> Directory

- `sync` action in Jinks
  - writes to dir on server machine
  - *docker*: inside container!
- `Download` .xar and unpack
- `Files` tab in Jinks (right click on file)

Sync Options: Directory -> eXist-db

- eXist-db plugin for Visual Studio Code and similar
- always **rebuild** `.xar` with `ant`
- use xst command line tool

Golden Rules

- always commit `.jinks.json` and `context.json`
- never change `context.json` (it's autogenerated every time)

Upgrading an old app (before v10)

There's no direct update path – app needs to be recreated.

1. **Create** new app in Jinks
   - **abbreviation** and **Unique Identifier** must be different
   - **add profiles** as needed, but start with minimum set first
2. **Download** the `.xar` package and unpack into new folder

Copy data

1. copy any data from the source app's `data` subdirectory into the target app's `data`
2. compare `collection.xconf` files and check if changes are necessary

Copy ODDs

1. copy any custom ODD, excluding `teipublisher.odd`, `tei_simplePrint.odd`, `docx.odd`, `teipublisher_odds.odd`
2. make sure that all custom ODDs are registered in `config.json`

```
{
    "odds" : [ "shakespeare.odd" ],
    "defaults" : {
        "odd" : "shakespeare.odd"
    }
}
```

3. Run **Fix ODDs** action to regenerate ODD configuration

Repair ODDs

1. check ODDs extending `teipublisher.odd` (`source` attribute on `<schemaSpec>`):
   - for each `<elementSpec>` overwriting one in `teipublisher.odd`: make sure that the `mode` attribute on the `<elementSpec>` is set to `"change"`, not `"add"`
2. check where `$parameters?header='short'` is used in a `predicate` attribute on a `model`. Replace with `$parameters?display='browse'`

Copy and adjust HTML templates

- Make sure `defaults.template` in `config.json` references correct template
- HTML templates need to be adjusted to new templating system
- Start from `templates/pages/basic.html` for document views
- Observe template inheritance hierarchy!